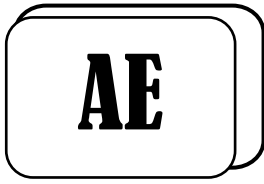




POLITEKNIK MANUFAKTUR BANDUNG

Jalan Kanayakan 21 Tlp/Fax. +62 (022) 7312564 Bandung 40275
Telp. (022)2500241 Faks. (022)2502649 <https://polman-bandung.ac.id/>



LAPORAN TUGAS MANDIRI 2 PEMROGRAMAN BERBASIS OBJEK (Minggu ke-1)



Nama : Syaroful Haidar Sudaryono
NIM : 225443051
Kelas : 2AEC2



DAFTAR ISI

DAFTAR ISI.....	1
PRAKTIKUM HARI KE – 2.....	2
(Sistem Manajemen Transportasi Online).....	2
1. Tujuan dan Penjelasan Kelas.....	2
2. Hasil Eksekusi (Demonstrasi Program).....	3
3. Analisis OOP.....	4
1.3. Kelas Dasar & Enkapsulasi	4
2.3. Pewarisan (Inheritance) & Polimorfisme	4
3.3. Interface & Sealed Class	4
LAMPIRAN	5
1. LAMPIRAN SOURCE CODE.....	5



PRAKTIKUM HARI KE – 2

(Sistem Manajemen Transportasi Online)

1. Tujuan dan Penjelasan Kelas

- **Tujuan Soal:** Mengimplementasikan konsep dasar OOP, Enkapsulasi, Pewarisan, Polimorfisme, dan integrasi sistem secara bertahap pada studi kasus Transportasi Online.
- **Penjelasan Kelas:** Kelas Vehicle beserta turunannya (Car, Motorcycle, Truck) mengelola data dan kalkulasi tarif. Driver dan Customer menyimpan entitas pengguna. Order menghubungkan pengguna, rute, dan status pesanan (menggunakan *sealed class* OrderStatus). Payment dan *interface* PaymentMethod menangani transaksi. Seluruh entitas ini dikelola secara terpusat oleh kelas TransportSystem.



2. Hasil Eksekusi (Demonstrasi Program)

```
=== Go-Transport 2024 ===

=== TARIF KENDARAAN UNTUK JARAK 20 KM ===
Mobil: Rp58.000
Motor: Rp33.000
Truk: Rp80.000

=== DAFTAR KENDARAAN ===
Nomor polisi : B 1234 XYZ
Kendaraan   : Toyota Innova (2021)
Jenis       : Mobil
Tersedia    : Ya
Bahan bakar : Bensin
Jumlah pintu : 4
|
Nomor polisi : D 5678 ABC
Kendaraan   : Honda Beat (2022)
Jenis       : Motor
Tersedia    : Ya
Kapasitas   : 125 cc
Helm        : Tersedia
|
Nomor polisi : E 9012 DEF
Kendaraan   : Hino Dutro (2020)
Jenis       : Truk
Tersedia    : Ya
Kapasitas   : 5.0 ton
Jumlah sumbu : 2

=== DAFTAR PENGEMUDI ===
ID driver   : D001
Nama driver : Andi
Telepon     : 08123456789
Status aktif : Ya
Nomor polisi : B 1234 XYZ
Kendaraan   : Toyota Innova (2021)
Jenis       : Mobil
Tersedia    : Ya
Bahan bakar : Bensin
Jumlah pintu : 4
|
ID driver   : D002
Nama driver : Budi
Telepon     : 08129876543
Status aktif : Ya
Nomor polisi : D 5678 ABC
Kendaraan   : Honda Beat (2022)
Jenis       : Motor
Tersedia    : Ya
Kapasitas   : 125 cc
Helm        : Tersedia
|
ID driver   : D003
Nama driver : Citra
Telepon     : 08125678901
Status aktif : Ya
Nomor polisi : E 9012 DEF
```

```
Kendaraan   : Hino Dutro (2020)
Jenis       : Truk
Tersedia    : Ya
Kapasitas   : 5.0 ton
Jumlah sumbu : 2

=== DAFTAR PELANGGAN ===
ID pelanggan : C001
Nama         : Dewi
Telepon      : 08134567890
Email        : dewi@email.com
Saldo        : Rp100.000
|
ID pelanggan : C002
Nama         : Eko
Telepon      : 08135678901
Email        : eko@email.com
Saldo        : Rp50.000
|
ID pelanggan : C003
Nama         : Fani
Telepon      : 08136789012
Email        : fani@email.com
Saldo        : Rp200.000

=== DAFTAR PESANAN ===
ID pesanan   : ORD-001
Pelanggan    : Dewi
Pengemudi    : Andi
Rute         : Kampus A -> Mall B
Jarak        : 12.0 km
Total tarif  : Rp38.000
Status       : Menunggu
|
ID pesanan   : ORD-002
Pelanggan    : Eko
Pengemudi    : Budi
Rute         : Stasiun -> Kantor
Jarak        : 8.0 km
Total tarif  : Rp15.000
Status       : Menunggu
|
ID pesanan   : ORD-003
Pelanggan    : Fani
Pengemudi    : Citra
Rute         : Gudang -> Pelabuhan
Jarak        : 25.0 km
Total tarif  : Rp97.500
Status       : Menunggu
```



3. Analisis OOP

1.3. Kelas Dasar & Enkapsulasi

- **val vs var:** `val` bersifat *immutable* (tidak dapat diubah setelah inisialisasi, contoh: `plateNumber`), sedangkan `var` bersifat *mutable* (dapat diperbarui, contoh: `balance`).
- **private set vs private val:** `private set` memungkinkan data dibaca secara bebas dari luar kelas melalui *getter* publik, namun modifikasinya dibatasi hanya dari dalam kelas itu sendiri. Hal ini krusial untuk melindungi integritas status pembayaran (`_isPaid`) dan status pesanan.
- **Validasi:** Validasi metode pembayaran (misal pengecekan 16 digit kartu kredit) berfungsi sebagai gerbang keamanan untuk mencegah sistem memproses input data yang tidak valid.

2.3. Pewarisan (Inheritance) & Polimorfisme

- **open class & super:** Kelas `Vehicle` diubah menjadi open class karena secara bawaan kelas Kotlin bersifat final. Pemanggilan `super.displayInfo()` pada sub-kelas digunakan agar tidak perlu menulis ulang logika pencetakan data dasar dari kelas induk.
- **Polimorfisme pada calculateFare():** Metode ini di-*override* karena mobil, motor, dan truk memiliki parameter tarif dasar dan per kilometer yang berbeda. Konsep ini juga terlihat pada `displayAllVehicles()`, di mana program otomatis mengeksekusi metode spesifik sesuai wujud asli kendaraannya meski disimpan secara umum bertipe `Vehicle`.

3.3. Interface & Sealed Class

- **Interface:** `PaymentMethod` menggunakan *interface* karena sistem hanya membutuhkan standarisasi fungsi pemrosesan tanpa memerlukan penyimpanan data turunan (*state*).
- **Sealed Class:** `PaymentResult` dan `OrderStatus` menggunakan *sealed class* agar hasilnya bersifat pasti. Kompiler menyadari seluruh skenario (ekshaustif), serta mendukung penyimpanan parameter dinamis (seperti alasan pada status `Cancelled`).
- **Smart Casting (as?):** Operator `as?` digunakan untuk konversi tipe data secara aman. Apabila objek tidak sesuai, sistem mengembalikan `null` alih-alih menghentikan program secara paksa.



LAMPIRAN

1. LAMPIRAN SOURCE CODE

```
// =====  
// 1. Vehicle.kt  
// =====  
  
/**  
 * Class dasar yang merepresentasikan kendaraan pada sistem transportasi online.  
 */  
 * @property plateNumber nomor polisi kendaraan.  
 * @property brand merek kendaraan.  
 * @property model model kendaraan.  
 * @property year tahun kendaraan.  
 * @property isAvailable status ketersediaan kendaraan.  
 */  
open class Vehicle(  
    val plateNumber: String,  
    val brand: String,  
    val model: String,  
    val year: Int,  
    var isAvailable: Boolean = true  
) {  
  
    /**  
     * Mengembalikan jenis kendaraan.  
     */  
    * @return jenis kendaraan.  
    */  
    open fun getType(): String = "Kendaraan Umum"  
  
    /**  
     * Menghitung tarif perjalanan berdasarkan jarak.  
     */  
    * @param distanceKm jarak perjalanan dalam kilometer.  
    * @return total tarif perjalanan.
```



```
    */

    open fun calculateFare(distanceKm: Double): Double {
        return 5000 + (distanceKm * 2000)
    }

    /**
     * Menampilkan informasi kendaraan ke console.
     */
    open fun displayInfo() {
        println("Nomor polisi : $plateNumber")
        println("Kendaraan : $brand $model ($year)")
        println("Jenis : ${getType()}")
        println("Tersedia : ${if (isAvailable) "Ya" else "Tidak"}")
    }
}

// =====
// 2. Car.kt
// =====

/**
 * Class kendaraan mobil yang merupakan turunan dari [Vehicle].
 *
 * @property fuelType jenis bahan bakar yang digunakan.
 * @property numberOfDoors jumlah pintu mobil.
 */
class Car(
    plateNumber: String,
    brand: String,
    model: String,
    year: Int,
    val fuelType: String,
    val numberOfDoors: Int
) : Vehicle(plateNumber, brand, model, year) {
```



```
/**
 * Mengembalikan jenis kendaraan sebagai mobil.
 *
 * @return "Mobil".
 */
override fun getType(): String = "Mobil"

/**
 * Menghitung tarif khusus kendaraan mobil.
 *
 * @param distanceKm jarak perjalanan dalam kilometer.
 * @return total tarif mobil.
 */
override fun calculateFare(distanceKm: Double): Double {
    return 8000 + (distanceKm * 2500)
}

/**
 * Menampilkan informasi dasar kendaraan dan informasi khusus mobil.
 */
override fun displayInfo() {
    super.displayInfo()
    println("Bahan bakar : $fuelType")
    println("Jumlah pintu : $numberOfDoors")
}
}

// =====
// 3. Motorcycle.kt
// =====

/**
 * Class kendaraan motor yang merupakan turunan dari [Vehicle].
```



```
*  
* @property engineCapacity kapasitas mesin dalam cc.  
* @property hasHelmet menunjukkan ketersediaan helm.  
*/  
class Motorcycle(  
    plateNumber: String,  
    brand: String,  
    model: String,  
    year: Int,  
    val engineCapacity: Int,  
    val hasHelmet: Boolean  
) : Vehicle(plateNumber, brand, model, year) {  
  
    /**  
     * Mengembalikan jenis kendaraan sebagai motor.  
     */  
    * @return "Motor".  
    */  
    override fun getType(): String = "Motor"  
  
    /**  
     * Menghitung tarif khusus kendaraan motor.  
     */  
    * @param distanceKm jarak perjalanan dalam kilometer.  
    * @return total tarif motor.  
    */  
    override fun calculateFare(distanceKm: Double): Double {  
        return 3000 + (distanceKm * 1500)  
    }  
  
    /**  
     * Menampilkan informasi dasar kendaraan dan informasi khusus motor.  
     */  
    override fun displayInfo() {  
        super.displayInfo()
```



```
println("Kapasitas : $engineCapacity cc")
println("Helm : ${if (hasHelmet) "Tersedia" else "Tidak ada"}")
}
}

// =====
// 4. Truck.kt
// =====

/**
 * Class kendaraan truk yang merupakan turunan dari [Vehicle].
 *
 * @property loadCapacity kapasitas muatan dalam ton.
 * @property numberOfAxles jumlah sumbu roda.
 */
class Truck(
    plateNumber: String,
    brand: String,
    model: String,
    year: Int,
    val loadCapacity: Double,
    val numberOfAxles: Int
) : Vehicle(plateNumber, brand, model, year) {

    /**
     * Mengembalikan jenis kendaraan sebagai truk.
     */
    @return "Truk".

    override fun getType(): String = "Truk"

    /**
     * Menghitung tarif khusus kendaraan truk.
     */
}
```



```
* @param distanceKm jarak perjalanan dalam kilometer.
* @return total tarif truk.
*/
override fun calculateFare(distanceKm: Double): Double {
    return 10000 + (distanceKm * 3500)
}

/**
 * Menampilkan informasi dasar kendaraan dan informasi khusus truk.
 */
override fun displayInfo() {
    super.displayInfo()
    println("Kapasitas : $loadCapacity ton")
    println("Jumlah sumbu : $numberOfAxles")
}
}

// =====
// 5. Driver.kt
// =====

/**
 * Class yang merepresentasikan pengemudi beserta kendaraan yang digunakan.
 */
*
* @property id ID pengemudi.
* @property name nama pengemudi.
* @property phone nomor telepon pengemudi.
* @property vehicle kendaraan yang digunakan pengemudi.
* @property isActive status keaktifan pengemudi.
*/
class Driver(
    val id: String,
    val name: String,
    val phone: String,
```



```
    val vehicle: Vehicle,
    var isActive: Boolean = true
) {

    /**
     * Menampilkan data pengemudi dan kendaraan.
     */
    fun displayInfo() {
        println("ID driver : $id")
        println("Nama driver : $name")
        println("Telepon : $phone")
        println("Status aktif : ${if (isActive) "Ya" else "Tidak"}")
        vehicle.displayInfo()
    }

    /**
     * Mengecek apakah pengemudi dapat menerima order.
     */
    *
    * @return `true` jika driver aktif dan kendaraan tersedia.
    */
    fun acceptOrder(): Boolean {
        return isActive && vehicle.isAvailable
    }
}

// =====
// 6. Customer.kt
// =====

/**
 * Class yang merepresentasikan pelanggan.
 */
*
* @property id ID pelanggan.
* @property name nama pelanggan.
```



```
* @property phone nomor telepon pelanggan.
* @property email email pelanggan.
* @property balance saldo pelanggan.
*/
class Customer(
    val id: String,
    val name: String,
    val phone: String,
    val email: String,
    var balance: Double = 0.0
) {

    /**
     * Menampilkan data pelanggan.
     */
    fun displayInfo() {
        println("ID pelanggan : $id")
        println("Nama : $name")
        println("Telepon : $phone")
        println("Email : $email")
        println("Saldo : ${formatRupiah(balance)}")
    }

    /**
     * Menambah saldo pelanggan.
     */
    *
    * @param amount jumlah saldo yang ditambahkan.
    */
    fun topUp(amount: Double) {
        require(amount > 0) {
            "Nominal top up harus lebih dari 0."
        }

        balance += amount
    }
}
```



```
/**
 * Mengecek apakah saldo pelanggan mencukupi.
 *
 * @param amount nominal yang akan dibayar.
 * @return `true` jika saldo mencukupi.
 */
fun canPay(amount: Double): Boolean {
    return balance >= amount
}
}

/**
 * Mengubah nominal menjadi format rupiah sederhana.
 *
 * @param amount nominal yang akan diformat.
 * @return string nominal dalam format rupiah.
 */
fun formatRupiah(amount: Double): String {
    return "Rp%,d".format(amount.toLong()).replace(',', ',.')
}
}

// =====
// 7. OrderStatus.kt
// =====

/**
 * Sealed class yang mewakili seluruh kemungkinan status sebuah order.
 */
sealed class OrderStatus {

    /**
     * Menampilkan nama status.
     *

```



```
* @return deskripsi status.
```

```
*/
```

```
abstract fun display(): String
```

```
/**
```

```
* Mengecek apakah status sudah merupakan status akhir.
```

```
*
```

```
* @return `true` jika order selesai atau dibatalkan.
```

```
*/
```

```
open fun isFinal(): Boolean {
```

```
    return this is Completed || this is Cancelled
```

```
}
```

```
/**
```

```
* Status ketika order masih menunggu perjalanan dimulai.
```

```
*/
```

```
object Waiting : OrderStatus() {
```

```
    override fun display(): String = "Menunggu"
```

```
}
```

```
/**
```

```
* Status ketika perjalanan sedang berlangsung.
```

```
*/
```

```
object OnGoing : OrderStatus() {
```

```
    override fun display(): String = "Berjalan"
```

```
}
```

```
/**
```

```
* Status ketika perjalanan sudah selesai.
```

```
*/
```

```
object Completed : OrderStatus() {
```

```
    override fun display(): String = "Selesai"
```

```
}
```

```
/**
```



```
* Status ketika order dibatalkan.
```

```
*
```

```
* @property reason alasan pembatalan.
```

```
*/
```

```
data class Cancelled(  
    val reason: String  
) : OrderStatus() {  
  
    override fun display(): String {  
        return "Dibatalkan: $reason"  
    }  
}
```

```
}
```

```
// =====
```

```
// 8. Order.kt
```

```
// =====
```

```
/**
```

```
 * Class yang merepresentasikan pesanan perjalanan.
```

```
 *
```

```
 * @property id ID order.
```

```
 * @property customer pelanggan yang membuat order.
```

```
 * @property driver driver yang menerima order.
```

```
 * @property pickupLocation lokasi penjemputan.
```

```
 * @property destination lokasi tujuan.
```

```
 * @property distanceKm jarak perjalanan dalam kilometer.
```

```
 */
```

```
class Order(  
    val id: String,  
    val customer: Customer,  
    val driver: Driver,  
    val pickupLocation: String,  
    val destination: String,
```



```
    val distanceKm: Double
} {

/**
 * Status order.
 *
 * Setter dibuat private agar perubahan status hanya melalui method
 * yang disediakan oleh class.
 */
    var status: OrderStatus = OrderStatus.Waiting
    private set

/**
 * Total tarif perjalanan yang dihitung berdasarkan kendaraan driver.
 */
    val totalFare: Double = driver.vehicle.calculateFare(distanceKm)

/**
 * Mengubah status order dari Menunggu menjadi Berjalan.
 *
 * @return `true` jika perubahan berhasil.
 */
    fun startTrip(): Boolean {
        if (status is OrderStatus.Waiting) {
            status = OrderStatus.OnGoing
            return true
        }

        return false
    }

/**
 * Mengubah status order dari Berjalan menjadi Selesai.
 *
 * @return `true` jika perubahan berhasil.
```



```
*/
```

```
fun completeTrip(): Boolean {
```

```
    if (status is OrderStatus.OnGoing) {
```

```
        status = OrderStatus.Completed
```

```
        driver.vehicle.isAvailable = true
```

```
        return true
```

```
    }
```

```
    return false
```

```
}
```

```
/**
```

```
 * Membatalkan order selama status belum final.
```

```
 */
```

```
 * @param reason alasan pembatalan.
```

```
 * @return `true` jika pembatalan berhasil.
```

```
*/
```

```
fun cancelTrip(reason: String): Boolean {
```

```
    if (!status.isFinal()) {
```

```
        status = OrderStatus.Cancelled(reason)
```

```
        driver.vehicle.isAvailable = true
```

```
        return true
```

```
    }
```

```
    return false
```

```
}
```

```
/**
```

```
 * Menampilkan detail order.
```

```
 */
```

```
fun displayOrder() {
```

```
    println("ID pesanan : $id")
```

```
    println("Pelanggan : ${customer.name}")
```

```
    println("Pengemudi : ${driver.name}")
```

```
    println("Rute : $pickupLocation -> $destination")
```



```
println("Jarak : $distanceKm km")
println("Total tarif : ${formatRupiah(totalFare)}")
println("Status : ${status.display()}")
}
}
```

```
// =====
// 9. PaymentResult.kt
// =====
```

```
/**
 * Sealed class yang mewakili hasil proses pembayaran.
 */
sealed class PaymentResult {
```

```
    /**
     * Menampilkan hasil pembayaran.
     *
     * @return deskripsi hasil pembayaran.
     */
    abstract fun display(): String
```

```
    /**
     * Hasil ketika pembayaran berhasil.
     *
     * @property transactionId ID transaksi.
     * @property timestamp waktu transaksi.
     */
    data class Success(
        val transactionId: String,
        val timestamp: String
    ) : PaymentResult() {
```

```
        override fun display(): String {
```



```
        return "Berhasil. ID transaksi: $transactionId"
```

```
    }
```

```
}
```

```
/**
```

```
 * Hasil ketika pembayaran gagal.
```

```
 */
```

```
 * @property reason alasan pembayaran gagal.
```

```
 * @property errorCode kode error.
```

```
 */
```

```
data class Failed(  
    val reason: String,  
    val errorCode: Int  
) : PaymentResult() {
```

```
    override fun display(): String {
```

```
        return "Gagal. $reason (kode $errorCode)"
```

```
    }
```

```
}
```

```
/**
```

```
 * Hasil ketika pembayaran masih tertunda.
```

```
 */
```

```
object Pending : PaymentResult() {
```

```
    override fun display(): String {
```

```
        return "Tertunda"
```

```
    }
```

```
}
```

```
}
```

```
// =====
```

```
// 10. PaymentMethod.kt
```

```
// =====
```



```
/**
 * Interface yang menjadi standar untuk metode pembayaran.
 */
interface PaymentMethod {

    /**
     * Nama metode pembayaran.
     */
    val name: String

    /**
     * Memproses pembayaran.
     */
    *
    * @param amount nominal pembayaran.
    * @return hasil proses pembayaran.
    */
    fun processPayment(amount: Double): PaymentResult

    /**
     * Menghitung biaya layanan dari metode pembayaran.
     */
    *
    * @param amount nominal pembayaran.
    * @return biaya layanan.
    */
    fun getFee(amount: Double): Double = 0.0
}

// =====
// 11. CreditCard.kt
// =====

import java.time.LocalDateTime
```



```
/**
 * Metode pembayaran menggunakan kartu kredit.
 *
 * @property cardNumber nomor kartu kredit yang digunakan.
 */
class CreditCard(
    val cardNumber: String
) : PaymentMethod {

    init {
        require(
            cardNumber.length >= 16 &&
            cardNumber.all { it.isDigit() }
        ) {
            "Nomor kartu kredit harus terdiri dari minimal 16 digit."
        }
    }

    /**
     * Nama metode pembayaran.
     */
    override val name: String = "Kartu Kredit"

    /**
     * Menghitung biaya layanan kartu kredit sebesar 2 persen.
     *
     * @param amount nominal pembayaran.
     * @return biaya layanan.
     */
    override fun getFee(amount: Double): Double {
        return amount * 0.02
    }

    /**
     * Memproses pembayaran kartu kredit.
```



```
*  
* @param amount nominal pembayaran.  
* @return hasil transaksi.  
*/  
override fun processPayment(amount: Double): PaymentResult {  
    return PaymentResult.Success(  
        "CC-#{System.currentTimeMillis()}",  
        LocalDateTime.now().toString()  
    )  
}  
}
```

```
// =====  
// 12. QRIS.kt  
// =====
```

```
import java.time.LocalDateTime
```

```
/**  
 * Metode pembayaran menggunakan QRIS.  
 *  
 * @property qrcode kode QRIS yang digunakan.  
 */  
class QRIS(  
    val qrcode: String  
) : PaymentMethod {  
  
    init {  
        require(qrcode.length >= 10) {  
            "Kode QRIS harus memiliki minimal 10 karakter."  
        }  
    }  
}
```

```
/**
```



```
* Nama metode pembayaran.
```

```
*/
```

```
override val name: String = "QRIS"
```

```
/**
```

```
 * Menghitung biaya layanan QRIS sebesar 0,5 persen.
```

```
 *
```

```
 * @param amount nominal pembayaran.
```

```
 * @return biaya layanan.
```

```
*/
```

```
override fun getFee(amount: Double): Double {
```

```
    return amount * 0.005
```

```
}
```

```
/**
```

```
 * Memproses pembayaran QRIS.
```

```
 *
```

```
 * @param amount nominal pembayaran.
```

```
 * @return hasil transaksi.
```

```
*/
```

```
override fun processPayment(amount: Double): PaymentResult {
```

```
    return PaymentResult.Success(
```

```
        "QRIS-${System.currentTimeMillis()}",
```

```
        LocalDateTime.now().toString()
```

```
    )
```

```
}
```

```
}
```

```
// =====
```

```
// 13. Cash.kt
```

```
// =====
```

```
import java.time.LocalDateTime
```



```
/**
 * Metode pembayaran menggunakan uang tunai.
 */
class Cash : PaymentMethod {

    /**
     * Nama metode pembayaran.
     */
    override val name: String = "Tunai"

    /**
     * Mengembalikan biaya layanan tunai.
     */
    *
    * @param amount nominal pembayaran.
    * @return 0 karena tunai tidak memiliki biaya layanan.
    */
    override fun getFee(amount: Double): Double {
        return 0.0
    }

    /**
     * Memproses pembayaran tunai.
     */
    *
    * @param amount nominal pembayaran.
    * @return hasil transaksi.
    */
    override fun processPayment(amount: Double): PaymentResult {
        return PaymentResult.Success(
            "CASH-${System.currentTimeMillis()}",
            LocalDateTime.now().toString()
        )
    }
}
```



```
// =====  
// 14. Payment.kt  
// =====  
  
/**  
 * Class yang mengelola pembayaran sebuah order.  
 *  
 * @property order order yang akan dibayar.  
 * @property method metode pembayaran yang digunakan.  
 */  
class Payment(  
    val order: Order,  
    val method: PaymentMethod  
) {  
  
    /**  
     * Nominal dasar tagihan dari order.  
     */  
    val amount: Double = order.totalFare  
  
    /**  
     * Status pembayaran.  
     */  
    *  
    * Setter dibuat private agar tidak dapat diubah langsung dari luar.  
    */  
    var isPaid: Boolean = false  
    private set  
  
    /**  
     * Memproses pembayaran beserta biaya layanan metode yang dipilih.  
     */  
    *  
    * @param paidAmount nominal yang dibayarkan.  
    * @return hasil proses pembayaran.  
    */  
    fun processPayment(paidAmount: Double): PaymentResult {
```



```
val totalDue = amount + method.getFee(amount)
```

```
if (!isPaid && paidAmount >= totalDue) {
```

```
    val result = method.processPayment(amount)
```

```
    if (result is PaymentResult.Success) {
```

```
        isPaid = true
```

```
    }
```

```
    return result
```

```
    return PaymentResult.Failed(
```

```
        "Nominal pembayaran kurang dari tagihan ${formatRupiah(totalDue)}",
```

```
        400
```

```
    )
```

```
}
```

```
/**
```

```
 * Menampilkan detail pembayaran.
```

```
*/
```

```
fun displayPayment() {
```

```
    val totalDue = amount + method.getFee(amount)
```

```
    println("Tagihan dasar : ${formatRupiah(amount)}")
```

```
    println("Biaya layanan : ${formatRupiah(method.getFee(amount))}")
```

```
    println("Total tagihan : ${formatRupiah(totalDue)}")
```

```
    println("Metode : ${method.name}")
```

```
    println("Status Lunas : ${if (isPaid) "Sudah" else "Belum"}")
```

```
}
```

```
}
```



```
// =====  
// 15. TransportSystem.kt  
// =====  
  
/**  
 * Class utama yang mengelola seluruh data dalam sistem transportasi online.  
 */  
 * @property name nama sistem transportasi.  
 */  
class TransportSystem(  
    val name: String  
) {  
  
    private val vehicles = mutableListOf<Vehicle>()  
    private val drivers = mutableListOf<Driver>()  
    private val customers = mutableListOf<Customer>()  
    private val orders = mutableListOf<Order>()  
    private val payments = mutableListOf<Payment>()  
  
    /**  
     * Mengembalikan daftar kendaraan dalam bentuk read-only list.  
     */  
    val vehiclesList: List<Vehicle>  
        get() = vehicles.toList()  
  
    /**  
     * Menambahkan kendaraan ke sistem.  
     */  
    * @param vehicle kendaraan yang akan ditambahkan.  
    */  
    fun addVehicle(vehicle: Vehicle) {  
        vehicles.add(vehicle)  
    }  
}
```



```
/**
```

```
 * Menambahkan driver ke sistem.
```

```
 *
```

```
 * @param driver driver yang akan ditambahkan.
```

```
 */
```

```
 fun addDriver(driver: Driver) {
```

```
     drivers.add(driver)
```

```
 }
```

```
/**
```

```
 * Menambahkan customer ke sistem.
```

```
 *
```

```
 * @param customer customer yang akan ditambahkan.
```

```
 */
```

```
 fun addCustomer(customer: Customer) {
```

```
     customers.add(customer)
```

```
 }
```

```
/**
```

```
 * Mencari kendaraan berdasarkan nomor polisi.
```

```
 *
```

```
 * @param plate nomor polisi kendaraan.
```

```
 * @return kendaraan yang ditemukan atau `null`.
```

```
 */
```

```
 fun findVehicle(plate: String): Vehicle? {
```

```
     return vehicles.find {
```

```
         it.plateNumber.equals(plate, true)
```

```
     }
```

```
 }
```

```
/**
```

```
 * Mencari driver berdasarkan ID.
```

```
 *
```

```
 * @param id ID driver.
```

```
 * @return driver yang ditemukan atau `null`.
```



```
    */  
  
    fun findDriver(id: String): Driver? {  
        return drivers.find {  
            it.id.equals(id, true)  
        }  
    }  
  
    /**  
     * Mencari customer berdasarkan ID.  
     *  
     * @param id ID customer.  
     * @return customer yang ditemukan atau `null`.  
     */  
  
    fun findCustomer(id: String): Customer? {  
        return customers.find {  
            it.id.equals(id, true)  
        }  
    }  
  
    /**  
     * Membuat order baru dari customer dan driver yang tersedia.  
     *  
     * @param customerId ID customer.  
     * @param driverId ID driver.  
     * @param pickup lokasi penjemputan.  
     * @param dest lokasi tujuan.  
     * @param distance jarak perjalanan.  
     * @return order baru atau `null` jika data tidak ditemukan atau driver tidak tersedia.  
     */  
  
    fun createOrder(  
        customerId: String,  
        driverId: String,  
        pickup: String,  
        dest: String,
```



```
        distance: Double
```

```
    ): Order? {
```

```
        val customer = findCustomer(customerId)
```

```
        ?: return null
```

```
        val driver = findDriver(driverId)
```

```
        ?: return null
```

```
        if (!driver.acceptOrder()) {
```

```
            println("Driver tidak aktif atau kendaraan tidak tersedia.")
```

```
            return null
```

```
        }
```

```
        driver.vehicle.isAvailable = false
```

```
        val order = Order(
```

```
            id = "ORD-%03d".format(orders.size + 1),
```

```
            customer = customer,
```

```
            driver = driver,
```

```
            pickupLocation = pickup,
```

```
            destination = dest,
```

```
            distanceKm = distance
```

```
        )
```

```
        orders.add(order)
```

```
        return order
```

```
    }
```

```
    /**
```

```
     * Memproses pembayaran untuk sebuah order.
```

```
     *
```

```
     * @param orderId ID order.
```

```
     * @param method metode pembayaran.
```



```
* @param paidAmount nominal yang dibayarkan.
```

```
* @return hasil pembayaran.
```

```
*/
```

```
fun processPayment(
```

```
    orderId: String,
```

```
    method: PaymentMethod,
```

```
    paidAmount: Double
```

```
): PaymentResult {
```

```
    val order = orders.find {
```

```
        it.id == orderId
```

```
    } ?: return PaymentResult.Failed(
```

```
        "Order tidak ditemukan",
```

```
        404
```

```
)
```

```
    val payment = Payment(order, method)
```

```
    payments.add(payment)
```

```
    return payment.processPayment(paidAmount)
```

```
}
```

```
/**
```

```
* Menyelesaikan sebuah order.
```

```
*
```

```
* @param orderId ID order.
```

```
* @return `true` jika order berhasil diselesaikan.
```

```
*/
```

```
fun completeOrder(orderId: String): Boolean {
```

```
    val order = orders.find {
```

```
        it.id == orderId
```

```
    } ?: return false
```



```
        return order.completeTrip()
```

```
    }
```

```
    /**
```

```
     * Membatalkan sebuah order.
```

```
     *
```

```
     * @param orderId ID order.
```

```
     * @param reason alasan pembatalan.
```

```
     * @return `true` jika pembatalan berhasil.
```

```
    */
```

```
    fun cancelOrder(  
        orderId: String,  
        reason: String  
    ): Boolean {  
  
        val order = orders.find {  
            it.id == orderId  
        } ?: return false  
  
        return order.cancelTrip(reason)  
    }  
  
    /**  
     * Menampilkan semua kendaraan yang terdaftar.  
     */  
    fun displayAllVehicles() {  
  
        println("=== DAFTAR KENDARAAN ===")  
  
        vehicles.forEach {  
            it.displayInfo()  
            println()  
        }  
    }  
}
```



```
/**
```

```
 * Menampilkan semua driver yang terdaftar.
```

```
*/
```

```
fun displayAllDrivers() {
```

```
    println("=== DAFTAR PENGEMUDI ===")
```

```
    drivers.forEach {
```

```
        it.displayInfo()
```

```
        println()
```

```
    }
```

```
}
```

```
/**
```

```
 * Menampilkan semua customer yang terdaftar.
```

```
*/
```

```
fun displayAllCustomers() {
```

```
    println("=== DAFTAR PELANGGAN ===")
```

```
    customers.forEach {
```

```
        it.displayInfo()
```

```
        println()
```

```
    }
```

```
}
```

```
/**
```

```
 * Menampilkan semua order yang terdaftar.
```

```
*/
```

```
fun displayAllOrders() {
```

```
    println("=== DAFTAR PESANAN ===")
```

```
    orders.forEach {
```

```
        it.displayOrder()
```



```
println()
}
}

/**
 * Menghitung dan menampilkan total pendapatan
 * dari order yang selesai.
 */
fun displayRevenueReport() {

    val totalRevenue = orders
        .filter {
            it.status is OrderStatus.Completed
        }
        .sumOf {
            it.totalFare
        }

    println("=== LAPORAN PENDAPATAN ===")
    println(
        "Pendapatan pesanan selesai: " +
        formatRupiah(totalRevenue)
    )
}

// =====
// 16. Main.kt
// =====

/**
 * Fungsi utama untuk menjalankan seluruh demonstrasi tugas mandiri PBO.
 *
 * Demonstrasi mencakup class dan object, encapsulation, inheritance,
```



```
* polymorphism, interface, sealed class, smart casting, dan integrasi sistem.
```

```
*/
```

```
fun main() {
```

```
    val system = TransportSystem("Go-Transport 2024")
```

```
    val car = Car(
```

```
        "B 1234 XYZ",
```

```
        "Toyota",
```

```
        "Innova",
```

```
        2021,
```

```
        "Bensin",
```

```
        4
```

```
    )
```

```
    val motorcycle = Motorcycle(
```

```
        "D 5678 ABC",
```

```
        "Honda",
```

```
        "Beat",
```

```
        2022,
```

```
        125,
```

```
        true
```

```
    )
```

```
    val truck = Truck(
```

```
        "E 9012 DEF",
```

```
        "Hino",
```

```
        "Dutro",
```

```
        2020,
```

```
        5.0,
```

```
        2
```

```
    )
```

```
    system.addVehicle(car)
```

```
    system.addVehicle(motorcycle)
```



```
system.addVehicle(truck)
```

```
val driver1 = Driver(
```

```
    "D001",
```

```
    "Andi",
```

```
    "08123456789",
```

```
    car
```

```
)
```

```
val driver2 = Driver(
```

```
    "D002",
```

```
    "Budi",
```

```
    "08129876543",
```

```
    motorcycle
```

```
)
```

```
val driver3 = Driver(
```

```
    "D003",
```

```
    "Citra",
```

```
    "08125678901",
```

```
    truck
```

```
)
```

```
system.addDriver(driver1)
```

```
system.addDriver(driver2)
```

```
system.addDriver(driver3)
```

```
val customer1 = Customer(
```

```
    "C001",
```

```
    "Dewi",
```

```
    "08134567890",
```

```
    "dewi@email.com",
```

```
    100000.0
```

```
)
```



```
val customer2 = Customer(
```

```
    "C002",
```

```
    "Eko",
```

```
    "08135678901",
```

```
    "eko@email.com",
```

```
    50000.0
```

```
)
```

```
val customer3 = Customer(
```

```
    "C003",
```

```
    "Fani",
```

```
    "08136789012",
```

```
    "fani@email.com",
```

```
    200000.0
```

```
)
```

```
system.addCustomer(customer1)
```

```
system.addCustomer(customer2)
```

```
system.addCustomer(customer3)
```

```
println("=== ${system.name} ===")
```

```
println()
```

```
println("=== TARIF KENDARAAN UNTUK JARAK 20 KM ===")
```

```
println(
```

```
    "Mobil: " +
```

```
    formatRupiah(car.calculateFare(20.0))
```

```
)
```

```
println(
```

```
    "Motor: " +
```

```
    formatRupiah(motorcycle.calculateFare(20.0))
```

```
)
```



```
println(
```

```
    "Truk: " +
```

```
        formatRupiah(truck.calculateFare(20.0))
```

```
)
```

```
println()
```

```
system.displayAllVehicles()
```

```
system.displayAllDrivers()
```

```
system.displayAllCustomers()
```

```
val order1 = system.createOrder(
```

```
    "C001",
```

```
    "D001",
```

```
    "Kampus A",
```

```
    "Mall B",
```

```
    12.0
```

```
) ?: return
```

```
val order2 = system.createOrder(
```

```
    "C002",
```

```
    "D002",
```

```
    "Stasiun",
```

```
    "Kantor",
```

```
    8.0
```

```
) ?: return
```

```
val order3 = system.createOrder(
```

```
    "C003",
```

```
    "D003",
```

```
    "Gudang",
```

```
    "Pelabuhan",
```

```
    25.0
```

```
) ?: return
```



```
println("=== DATA AWAL ORDER ===")
```

```
system.displayAllOrders()
```

```
println("=== DEMONSTRASI STATUS ORDER ===")
```

```
println(
```

```
    "Start Order 1 : " +
```

```
    order1.startTrip()
```

```
)
```

```
println(
```

```
    "Complete Order 1 : " +
```

```
    system.completeOrder(order1.id)
```

```
)
```

```
println(
```

```
    "Cancel Order 1 setelah selesai : " +
```

```
    system.cancelOrder(
```

```
        order1.id,
```

```
        "Tidak jadi"
```

```
)
```

```
)
```

```
println()
```

```
println("=== PEMBAYARAN ORDER 1 ===")
```

```
val qris = QRIS("12345678901234")
```

```
val qrisTotal =
```

```
    order1.totalFare +
```

```
    qris.getFee(order1.totalFare)
```

```
val pay1 = system.processPayment(
```



```
order1.id,
```

```
gris,
```

```
grisTotal
```

```
)
```

```
println(pay1.display())
```

```
println()
```

```
println("=== PEMBAYARAN ORDER 2 ===")
```

```
val cash = Cash()
```

```
val pay2 = system.processPayment(
```

```
order2.id,
```

```
cash,
```

```
10000.0
```

```
)
```

```
println(
```

```
"Pembayaran tunai kurang: " +
```

```
pay2.display()
```

```
)
```

```
customer2.topUp(20000.0)
```

```
println(
```

```
"Saldo Eko setelah top up: " +
```

```
formatRupiah(customer2.balance)
```

```
)
```

```
val creditCard =
```

```
CreditCard("1234567890123456")
```

```
val creditCardTotal =
```



```
order2.totalFare +
```

```
creditCard.getFee(order2.totalFare)
```

```
val pay3 = system.processPayment(
```

```
order2.id,
```

```
creditCard,
```

```
creditCardTotal
```

```
)
```

```
println(
```

```
"Pembayaran kartu kredit: " +
```

```
pay3.display()
```

```
)
```

```
println()
```

```
println("=== PEMBATALAN ORDER 3 ===")
```

```
println(
```

```
"Order 3 dibatalkan: " +
```

```
system.cancelOrder(
```

```
order3.id,
```

```
"Hujan deras"
```

```
)
```

```
)
```

```
println()
```

```
println("=== STATUS AKHIR ORDER ===")
```

```
system.displayAllOrders()
```

```
system.displayRevenueReport()
```

```
println()
```



```
println("=== DEMONSTRASI SOAL 4 ===")
```

```
val methods: List<PaymentMethod> = listOf(  
    CreditCard("1234567890123456"),  
    QRIS("12345678901234"),  
    Cash()  
)
```

```
methods.forEach { method ->
```

```
    val result =
```

```
        method.processPayment(50000.0)
```

```
    when (result) {
```

```
        is PaymentResult.Success ->
```

```
            println(  
                "${method.name}: " +  
                result.display()  
            )
```

```
        is PaymentResult.Failed ->
```

```
            println(  
                "${method.name}: " +  
                result.display()  
            )
```

```
        PaymentResult.Pending ->
```

```
            println(  
                "${method.name}: " +  
                result.display()  
            )
```

```
    }
```

```
}
```



```
println()
```

```
println("=== DEMONSTRASI POLIMORFISME ===")
```

```
val vehicleList: List<Vehicle> =
```

```
    listOf(
```

```
        car,
```

```
        motorcycle,
```

```
        truck
```

```
    )
```

```
vehicleList.forEach { vehicle ->
```

```
    println(
```

```
        "${vehicle.getType()} " +
```

```
        "${vehicle.plateNumber}: " +
```

```
        formatRupiah(
```

```
            vehicle.calculateFare(15.0)
```

```
        )
```

```
    )
```

```
    when (vehicle) {
```

```
        is Car ->
```

```
            println(
```

```
                "Bahan bakar : " +
```

```
                vehicle.fuelType
```

```
            )
```

```
        is Motorcycle ->
```

```
            println(
```

```
                "Kapasitas mesin : " +
```

```
                vehicle.engineCapacity +
```

```
                " cc"
```



```
    )

    is Truck ->
    println(
        "Kapasitas muatan : " +
        vehicle.loadCapacity +
        " ton"
    )
}

val safeCast =
    driver1.vehicle as? Car

println(
    "\nHasil casting aman kendaraan D001: " +
    (
        safeCast?.fuelType
        ?: "Bukan Mobil"
    )
)

println()

println("=== DEMONSTRASI SEALED CLASS ===")

val statuses: List<OrderStatus> =
    listOf(
        OrderStatus.Waiting,
        OrderStatus.OnGoing,
        OrderStatus.Completed,
        OrderStatus.Cancelled(
            "Perubahan rencana"
        )
    )
}
```



```
statuses.forEach { status ->
```

```
    when (status) {
```

```
        is OrderStatus.Waiting ->
```

```
            println(status.display())
```

```
        is OrderStatus.OnGoing ->
```

```
            println(status.display())
```

```
        is OrderStatus.Completed ->
```

```
            println(status.display())
```

```
        is OrderStatus.Cancelled ->
```

```
            println(status.display())
```

```
    }
```

```
println()
```

```
println("=== DEMONSTRASI ENKAPSULASI ===")
```

```
println(
```

```
    "Status order hanya dapat diubah " +
```

```
    "melalui method startTrip(), " +
```

```
    "completeTrip(), atau cancelTrip()." +
```

```
)
```

```
println(
```

```
    "Akses langsung terhadap setter status " +
```

```
    "dibatasi karena menggunakan private set." +
```

```
)
```

```
}
```